ECE2072 Project

Team 77

Name:
Wu Li Min (34478841)
Yeoh Jie Ying (34601643)

Table of content

1.0 Introduction **2**
2.0 (Task 1) Component Overview, Functionality and Design Choices **3**
3.0 (Task 1) Resource Usage and Timing Analysis **5**
4.0 (Task 2/3) Two Instruction Datapath Implementation Example **6**
5.0 Design of Testing Regime (All Tasks) **8**
6.0 Conclusion and Justification **12**
7.0 Appendix **13**

1.0 Introduction

A computer processor, or central processing unit (CPU) is an important hardware component responsible for executing instructions in a computing system. It has the following operations: to obtain input data, decode the instructions, execute arithmetic calculations and other logical operations, and write back the output.

This project aims to design a simplified x72 processor implemented using verilog language and on FPGA hardware stimulation. It has two types of instructions, Type 1 register to register instructions and Type 2 immediate instructions. The processor is constructed from five essential components - sign extender, FSM tick counter, arithmetic logic unit (ALU), multiplexer and register.

The processor design is as follows. When the tick counter is enabled, the processor would progress through four states using one-hot encoding. In Tick 1, the processor receives a 9-bit instruction input (DIN) that is extended to 16-bit using the sign extender module for bus compatibility. Then, the control unit would decode and execute the instruction (addition, subtraction, multiplication, signed shift immediate, display, move and add immediate) based on the opcode. In Tick 2, for Type 1 instructions, the Bus Control would load the appropriate register value onto the bus, whereas for Type 2 instructions, the immediate value is loaded. After that, the bus value would be shifted into register A for ALU input or register H for HEX output. In Tick 3, if ALU is required, the second operand is placed on the bus, and the ALU performs arithmetic operations and stores the result in register G. Lastly, in Tick 4, the multiplexer allows output in register G back to the bus, and allows results to be written onto the destination register (R0-R7). For move immediate and display function, both tick 3 and 4 would be idle.

This design allows the bus to interconnect all components in the processor effectively, which enables less resources and high coordination between different components. All operations are synchronized using clock, enable and reset signals to control the processor. A testbench is then created to ensure all functions are working correctly.

Finally, the last task integrates all individual components into a complete processor datapath that allows the processor to execute instructions automatically one after another. The design introduces a program counter (PC) and branch logic supported by an instruction memory.

2.0 (Task 1) Component Overview, Functionality and Design Choices

1) Sign Extender

Functionality: The sign extender module converts the 9-bit instruction input (DIN) obtained from the switches into a 16-bit output to match the bus data width. This ensures that the data output is a signed 2's complement representation binary number. If the input is negative (MSB = 1), the upper seven bits [15:9] would be added with 1's, vice versa, if the input is positive (MSB = 0), they would be added with 0's.

Design Choices: This is a behavioral design and uses procedural assignment with the combinational always@(*) block. It also uses the if…else structure to assign bits based on positive or negative values. This approach allows the function to respond whenever there are input changes.

2) Tick Counter

Functionality: The tick counter acts as a one-hot encoded finite state machine (FSM) that transitions between four states (0001, 0010, 0100, 1000) and wraps back to the original state again. When the reset is asserted, the FSM would return to its initial state. When enabled = 1, the FSM transitions to the next state on each positive clock edge, executing the processor instruction from tick 1 to 4.

Design Choices: This is a behavioral design with synchronization using always @(posedge clk) block. The module uses the if…else statement to determine the output based on reset or enable, with a case statement in the enable block to manage state transition. One-hot encoding reduces combinational logic which leads to faster state transition and shorter delays.

3) ALU

Functionality: The ALU module performs different arithmetic operations based on a 3-bit control input (alu_op) with two inputs A and B. The operations included are 000 - multiplication, 001 - addition, 010 - subtraction, 011 - signed shift input B by A, and no operation for the rest control input.

Design Choices: This is a behavioral combinational logic with the always@(*) block and a case statement to determine the appropriate operation performed for different control inputs. The syntax $signed is used for signed shift operations.

4) **Multiplexer**

Functionality: The multiplexer module manages the 16-bit bus by selecting one of ten possible inputs through a 4-bit select line, which are the sign-extended DIN, register R0-R7 or register G (ALU output). The selected value would then be loaded onto the bus for future uses.

Design Choices: This is a behavioral design which uses the always@(*) block and a case statement to choose the appropriate register to load data from the bus. The multiplexer ensures the data could be shared internally across different components efficiently.

5) **Register**

Functionality: Each register stores the value from the bus and outputs it for use by other components. When reset is asserted, the register value clears all back to 0. When enable = 1, the register would shift in the input value at each positive edge clock and store for future uses.

Design Choices: This behavioral synchronous design is implemented using the always @(posedge clk) line and if…else statement to handle conditions for reset and enable. It also uses the parameter N that allows flexible register width (9-bit IR and 16-bit register) for code reusability.

3.0 (Task 1) Resource Usage and Timing Analysis

Module	Timing Constraint (MHz)	Logic Usage
Sign extend	1094.09	26 Logic Element + 0 DSPs
Tick FSM counter	510.46	12 Logic Element + 0 DSPs
ALU	152.44	240 Logic Element + 2 DSPs
Multiplexer	242.31	190 Logic Element + 0 DSPs
Register	562.75	51 Logic Element + 0 DSPs

The sign extend module achieves the highest maximum frequency of 1094.09MHz, indicating that it has minimum propagation delay as the design applies purely combinational logic and only performs bit replication. The small logic usage of 26 logic elements further confirms the efficiency of the module. (Please refer to **Appendix 7.2 Figure 7.2.1** for full Quartus timing report screenshots.)

The tick FSM counter operates at 510.46 MHz as it is limited by the presence of flip flops using the one-hot encoding implementation. These sequential logic introduce moderate propagation delay between clock cycles. (Please refer to **Appendix 7.2 Figure 7.2.2** for full Quartus timing report screenshots.)

The ALU shows the lowest frequency at 152.44 MHz due to longer time when performing arithmetic operations, especially multiplication. The logic usage of 240 logic elements and 2 DSPs used reflects the multiplier resource usage which slows down the design.(Please refer to **Appendix 7.2 Figure 7.2.3** for full Quartus timing report screenshots.)

The multiplexer module uses many logic elements as it handles ten 16-bit inputs loading the value onto the bus. Its maximum frequency of 242.31 MHz indicates moderate propagation delays affected by handling wide input and selection logic. (Please refer to **Appendix 7.2 Figure 7.2.4** for full Quartus timing report screenshots.)

The register sequential design that uses flip-flops allows fast data shifting across registers which uses minimal resources. This results confirms the register behavior for data storage at fast clock speeds.(Please refer to **Appendix 7.2 Figure 7.2.5** for full Quartus timing report screenshots.)

4.0 (Task 2/3) Two Instruction Datapath Implementation Example

To allow transition to the next FSM tick, enable is asserted and positive clock edge is applied after each tick. The register module is responsible for loading the current bus value onto the designated register by allowing enable = 1. The opcodes are as follows, 000-disp, 001-add, 010-addi, 011-sub, 100-mul, 101-ssi and 111-movi.

First Instruction: Addition
The first instruction is to perform addition, which is a register to register instruction (type 1). For example, add R3 containing the value of 10 and R2 of value of 3 and store the result back to R3. R3 has the register number of 011 and R2 of 010.

Tick 1:
Input a 9-bit instruction DIN (opcode = 001, rx = 011, ry = 010) into the instruction register (IR). The sign extender module would extend the 9-bit to 16-bit for bus compatibility. Then, the control unit would allow IR_in = 1 to fetch the data so that the control unit can decode opcode = 001 as ADD and execute the instruction.

Tick 2:
Based on the rx value (rx = 011), the Bus Control would select R3 (0100) current data and load it onto the bus. As this is an ALU instruction, the R3 value (10) is loaded into register A by allowing A_in = 1 for temporary storage.

Tick 3
Based on the ry value (ry = 010), the Bus Control now selects R2 (0011) to drive its current data onto the bus. Using the opcode 001, the ALU would perform addition A + Bus (R3 +R2) by allowing alu_op = 001 and store the result (10 + 3 = 13) into register G by allowing G_in = 1.

Tick 4
The Bus Control now selects register G (1001) output to load onto the bus. The R3_in is asserted to load the bus value into the R3 register. Then, the FSM wraps back to tick 1 waiting for the next instruction.

Second instruction: Signed Shift Immediate (SSI)
The second instruction is to perform signed shifting of input B by input A to the left for positive shift bits and to the right for negative shift bits, which is an immediate value operation (type 2). For example, shift R0 with value of 8 by the immediate value of 3 and store the result back to R0. R0 has the register number of 000.

Tick 1:
Load a 9-bit instruction DIN (opcode = 101, rx = 000, ry = 000) into the instruction register by allowing IR_in = 1 which is then sign-extended to 16 bit. Later, the control unit would decode opcode = 101 as the SSI instruction.

Tick 2:
Inputting the immediate value (3) from DIN (000000011), the Bus Control would select (0000) to load the sign-extended immediate value onto the bus. Then, the bus value is loaded into register A by allowing A_in = 1 for temporary storage as input A in the ALU module. Now, A = 3.

Tick 3:
Based on the rx value (rx = 000), the Bus Control now selects R0 (0001) to drive its current data 8 onto the bus. Using the opcode 101, the ALU would perform a positive signed shift by 3 bits to the left by allowing alu_op = 011 and store the result (8 << 3 = 64) into register G by allowing G_in = 1.

Tick 4:
The Bus Control now selects register G (1001) output to load onto the bus. The R0_in is asserted to load the shifted value of register G into the R0 register. Finally, the FSM wraps back to tick 1 waiting for the next instruction.

5.0 Design of Testing Regime (All Tasks)

Task 1:

Assumptions & Configurations:

- Uses one global clock with posedge triggering to drive all DUTs one testbench module
- Test cases of a combination of self-checking and manually computed cases

1) **Sign extender testbench**
   The sign extender testbench loop through all 512 possible 9-bit inputs. The testbench computes {{7{in[8]}}, in}) to check if the dut result matches exactly. This method covers all possible input, including the condition when the output is padded with 1's if the input is negative (MSB = 1), and 0's if the input is positive (MSB = 0) for 2's complement number. Upon encountering any error, the testbench would display "Error: in=**9-bit input**. Got ext=**actual 16-bit output**. Expected=**expected output** at **time**". A final summary would state 512 test cases checked and the number of errors it encountered.

2) **Tick FSM counter testbench**
   The tick FSM testbench verifies the one-hot encoding sequence after release reset and enable = 1. Using a case statement, the testbench ensures the tick progress form 0001, 0010, 0100, 1000 and wraps back again to the start of the sequence at each positive clock edge over two full cycles. Each tick compares the actual dut and expected tick, while an error message would display "Error : expected **tick**, got **actual tick** at **time**". A summary would print a total of 8 checks and number of errors.

3) **Multiplexer**
   The multiplexer testbench uses a helper function expected_result_mux(sel) to assign each select line to their respective destination (SIgnExtDin, R0-7 and G). It then assigns a distinct initial value to all sources and compares the value of the current bus and expected register content. It runs through all 16 (4-bit) select lines, valid or invalid to ensure the registers are wired correctly. The error message would print "Error : sel=**4-bit select line**, Got=**Bus value**, Expected=**Expected value** at **time"**, with a printed summary of 16 test cases run.

4) **ALU**
   The ALU testbench covers the checking of four functions, including mul (000), add (001), sub (010) and shift (011). By assigning four different inputs for each A and B randomly, the testbench loops through all alu_op containing four arithmetic operations between different possible combinations of values of A and B (4 * 4 * 8). Other than that, there are 5 handpicked manually computed boundary cases, including addition and multiplication overflow, subtraction underflow, and positive and negative value shifting. The error message displayed would print "Error : alu_op=**op**, input A =**A**, input B =**B**, Got=**Actual output**, Expected=**Expected output** at **time**", with a final summary of 133 tests run.

5) **Register**
   The register testbench tested 5 conditions covering all functions in the module. We test reset function to ensure all register content are back to 0, release reset and r_in = 1 to load input, r_in = 0 for the register to hold output value despite input changes, r_in again to ensure register content of 656 possible input are shifted correctly, and finally reset again to ensure register content are all cleared. The error message would display "Error: when **condition**, Got Q=**Actual register value**, Expected= **Expected register value**" with a summary of 660 tests run and error count.

After all components in the testbench are done, a conclusion is printed with all summary results. Please refer to **Appendix 7.3.1.2** for components testbench waveform and transcript.

Task 2

Assumptions & Configurations:

- Uses one global clock with posedge triggering to drive all DUTs
- A combination of self-checking sequences and manually computed edge cases
- Using timescale of 1ns/1ns

Helper functions

1) A sign_extender function is used to replicate the DUT 9-bit input to 16-bit output for bus compatibility.
2) Combine_din function is used to encode instructions by concatenating opcode, rx and ry into a 9 bit DIN.
3) The bcd_7 function is responsible for converting each decimal digit to their respective active-low hex display using a different combinational logic in the BCD module.
4) The bcd7_to_digit is used to convert the seven segment hex display into their respective decimal digits for self-checking.
5) The task start_with_tick1 is used to ensure the tick counter is at tick 1 before every test case checking.
6) The task run_ticks_and_check_hex5 is responsible for progressing through all 4 ticks while checking for HEX5 displays 1,2,3,4 in order. On mismatch, an error message indicating "Error (HEX5 tick): sub-tick **tick** Got=**DUT tick** Expected=**TB tick** (digit **TB tick decimal)**" is displayed.
7) The task run_imm_ticks is used for immediate instructions that drive the opcode at tick 1, then the immediate at tick 2.
8) The function get_register is used to connect testbench internal register (tR), set_register to assign testbench output value after arithmetic operation and get_real_register to obtain the DUT output register (R).

The proc testbench checks through all functions and components of the task 2 module by comparing the expected TB state with the DUT state. First, the reset function is tested to ensure all register content reads 16'h0000 when rst = 1 and enable = 0. Upon any errors, the message would print "Error: when rst=1, all register content should reset to 0. Got R7=**DUT output**, Expected=0000".

Secondly, the testbench checks the MOVI function. For three loops (change = 0, 1, 2), it iterates through all registers Rx (R0-7) and drives {111, Rx, don't care}. The testbench then presents the immediate value on the next tick using sign-extended {111, Rx, change+1} to allow distinct values for each register every cycle. After progressing through all 4 ticks using the helper function, it then compares tR[Rx] (testbench register) and R[Rx] (DUT register). The MOVI test covers all 24 cases (3 loops * 8 register) with varied immediates and verify sign extension function and correct writeback for each register. Error message would display "Error: when MOVI: loop=**change** Rx=**register** old=**last rx**, imm(signext(din=**imm**)= **immediate**. Got=**DUT value** Expected=**TB value**".

Thirdly, the testbench checks the ADD function. For three loops, iterate through all registers (Rx = 0-7, Ry - 0-7) to perform addition on all possible combinations of pairs of registers, capturing different values for each register after each addition. The testbench drives DIN = {001, Rx, Ry} and the expected tR[Rx] = Rx + Ry after 4 ticks. The testbench covers 192 (3 * 8 * 8) test cases and confirm the actual DUT addition and testbench addition are compatible. Upon any error, the message would display "Error: when ADD: loop=**change** Rx=**register** old=**last rx** + Ry=**register** old=**last ry**. Got Rx=**DUT value** Expected=**TB value**".

Fourthly, the testbench checks the SUB function. With the same datapath as ADD, for three loops, iterate through all registers (Rx = 0-7, Ry - 0-7) to perform subtraction on all possible combinations of pairs of registers. The testbench drives DIN = {011, Rx, Ry} and the expected tR[Rx] = Rx - Ry after 4 ticks. The testbench also covers 192 (3 * 8 * 8) test cases by comparing actual and expected register value after each subtraction. The error message would display "Error: when SUB: loop=**change** Rx=**register** old=**last rx** - Ry=**register** old=**last ry**. Got Rx=**DUT value** Expected=**TB value**".

Fifth, the testbench checks the ADDI function using the helper function run_imm_ticks. By looping through three loops, and for each Rx (R0-7) register, the testbench drives DIN = {010, Rx, don't care} for the first tick. At the second tick, the testbench drives in distinct sign-extended immediate value {6'd0, change + 1} each cycle and performs addition on the respective register. The testbench covers a total of 24 (3 * 8) test cases and sufficiently compare the actual and expected register values. The error message, if any, would then display "Error: when ADDI: loop=**change** Rx=**register** old=**last rx** + imm(signext=**imm**). Got=**DUT value** Expected=**TB value**.

Sixth, the testbench checks the HEX5 display by ensuring that it will progress through 1, 2, 3 and 4 in order with an active-low seven segment display for each instruction executed. The testbench compares the actual DUT and expected testbench hex display. The error message would display "Error (HEX5 tick): sub-tick **tick** Got=**DUT HEX** Expected=**TB HEX** (digit **TB digit**)".

Lastly, the testbench checks for 2 boundary edge cases computed manually. The testbench checks for addition overflow (16d'32767 + 16d'1 -> 16'h8000) where it would truncate the most significant bit, and for subtraction underflow (16'd-32768 - 16'd1 -> 16'h7FFF).

Please refer to **Appendix 7.3.2.2** for components testbench waveform and transcript.

Task 3

Assumptions & Configurations:

- Uses one global clock with posedge triggering to drive all DUTs
- A combination of self-checking sequences and manually computed edge cases
- Using timescale of 1ns/1ns
- Addition upon task 2 testbench

Helper functions (additions beyond task 2)

1) The bcd_8 function is used to add another signed decimal place by the extended BCD path.
2) The mod10_4 function is used to perform modulus operation while avoiding 32-bit division.
3) The split_digits task is to split a 16-bit signed value into five decimal digits with decimal place indicating negative value.
4) The check_hex_display task is responsible for checking the DUT and TB hex display of HEX0-4 for the expected decimal digits.

The proc_extension testbench reuse all Task 2 checks (Reset, HEX5 display, MOVI, ADD, SUB and ADDI). The following describes the added Task 3 coverage.

First, the testbench checks the MUL function. For three loops, iterate through all registers (Rx = 0-7, Ry - 0-7) to perform multiplication on all possible combinations of pairs of registers. The testbench drives DIN = {100, Rx, Ry} and the expected tR[Rx] = Rx * Ry after 4 ticks to compare between actual DUT and TB arithmetic results. The testbench covers all 192 (3 * 8 * 8) test cases over a range of values that naturally explores sign and wrap. Upon any error, the message would display "Error: when MUL: loop=**change** Rx=**register** old=**last rx** * Ry=**register** old=**last ry**. Got Rx=**DUT value** Expected=**TB value**".

Next, the testbench checks the SSI function using the helper function run_imm_ticks. By looping through three loops, and for each Rx (R0-7) register, the testbench drives DIN = {101, Rx, don't care} for the first tick. Then, at the next tick, the testbench drives in distinct sign-extended immediate value {6'd0, change + 1} each cycle and performs signed shifting on the respective register. In total, the testbench covers all 24 (3 * 8) test cases with different shift magnitudes, including negative operand tests from prior MOVI sequences. Any mismatch would display the error message "Error: when SSI: loop=**change** Rx=**register** old=**last rx** shift=**shift magnitude**. Got=**DUT value** Expected=**TB value**".

Other than that, the function DISP is checked to validate the BCD conversion to the five HEX digits with sign. For three loops and iterating through all Rx (R0-7), the testbench drives DIN = {000, Rx, don't care} to copy Rx to the display register. After progressing through all four ticks, the helper function check_hex_display is called, which functions to split 16-bit binary into 5 bit signed decimal digit, encodes each digit to the expected HEX values and finally compares against the DUT outputs, with each mismatch printing "Error (HEX**0**): Got=**DUT HEX** Expected=**TB HEX** (val=**TB decimal digit**)" and a total of 8 test cases run for each register.

Lastly, another three edge cases are computed manually, which includes checking for multiplication overflow ((-256)*(-256) = 16'h0000) where the overflow bits are truncated, positive value shift (+1 << 3 = +8) and negative value shift (-64 << 3 = -512).

Please refer to **Appendix 7.3.3.2** for components testbench waveform and transcript.

Justification:

The test cases are sufficient to prove the x72 processor is functioning correctly as

1) It has full coverage of all functions and components as instructed (disp, add, addi, sub, mul, ssi and movi). Each instruction is progressed through 4 ticks and checked against the expected result.
2) The testbench has cross-product operand coverage that tests all 8x8 register pairs across multiple loops which allows natural value variation. This also ensures individual components like sign extend, ALU, multiplexer, register and tick FSM are correct.
3) Manually computed edge cases are done to ensure the processor functions correctly even in boundary conditions.
4) Hardware testing on FPGA correctly stimulates all the functions and components of the processor.

6.0 Conclusion and Justification

The overall design of the x72 processor and its comprehensive testbenches demonstrates fully functional design for deployment. Each individual component (sign extend, tick FSM, ALU, multiplexer and register) functions correctly as a fundamental building block for the processor and passes all testbench test cases.

Moving onto task 2 and 3, both register to register (Type 1) and immediate register (Type 2) instructions execute correctly under the correct 4 tick sequence. The self-checking testbench also verifies all core instructions (MOVI, ADD, SUB, ADDI, MUL, SSI and DISP) are robust through systematic looping and direct comparison between DUT and testbench results with 3396 test cases.

As all modules pass all the test cases stimulated and on the DE-10 board, the processor is functioning correctly with synchronous timing and fit for reliable deployment in hardware implementation. The x72 processor successfully obtains data input, decodes and executes instructions, and writes back the result.

7.0 Appendix

7.1 Google Drive Links

https://drive.google.com/drive/folders/1WZFRqk89JEaNLEXFWxSptD34wil5Acyg?usp=sharing

7.2 Timing analysis of Task 1 component

7.2.1 Sign extend analysis

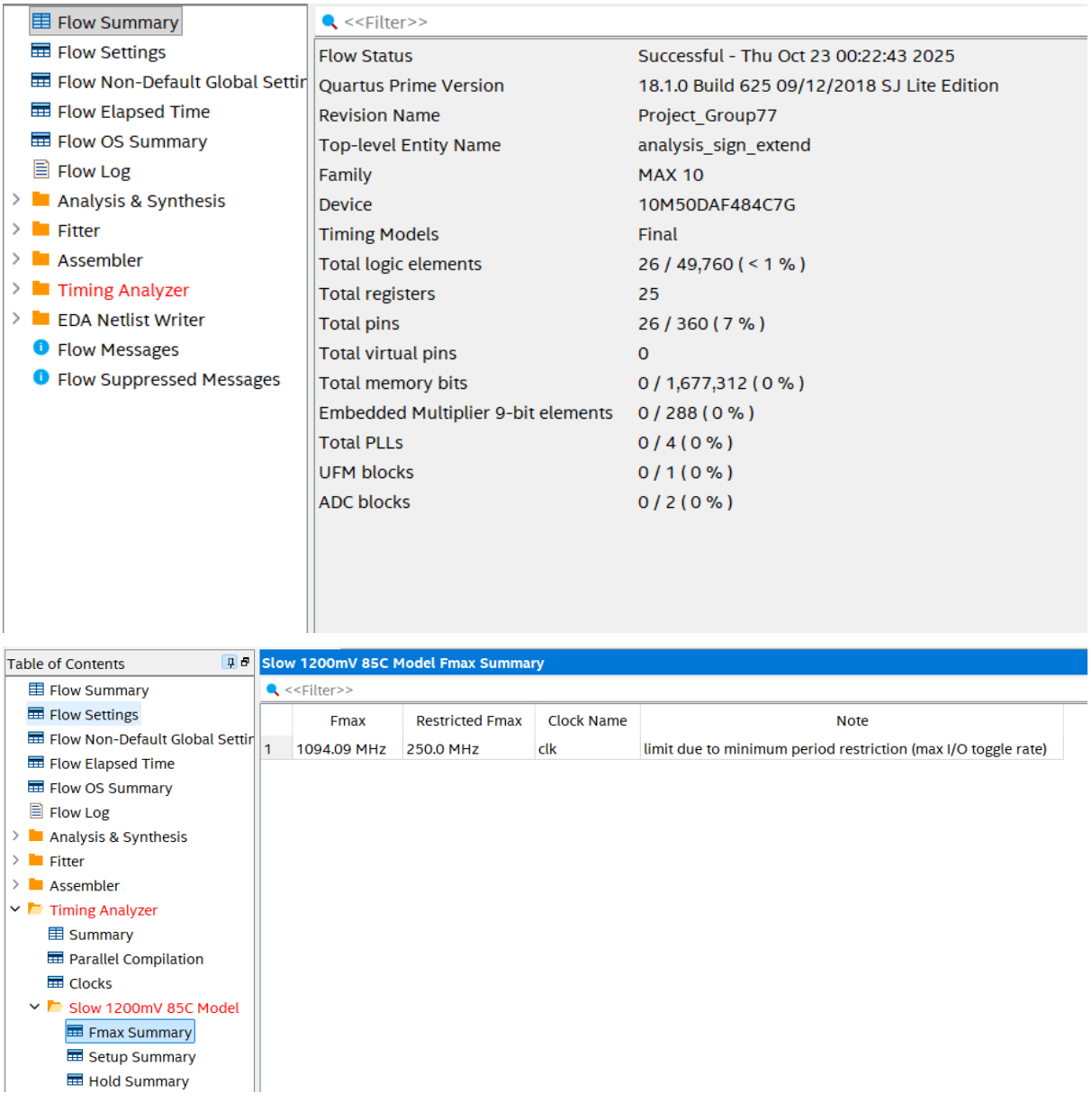


Figure 7.2.1: Logic Usage and Timing Constraints of Sign Extend module

7.2.2 Tick FSM analysis

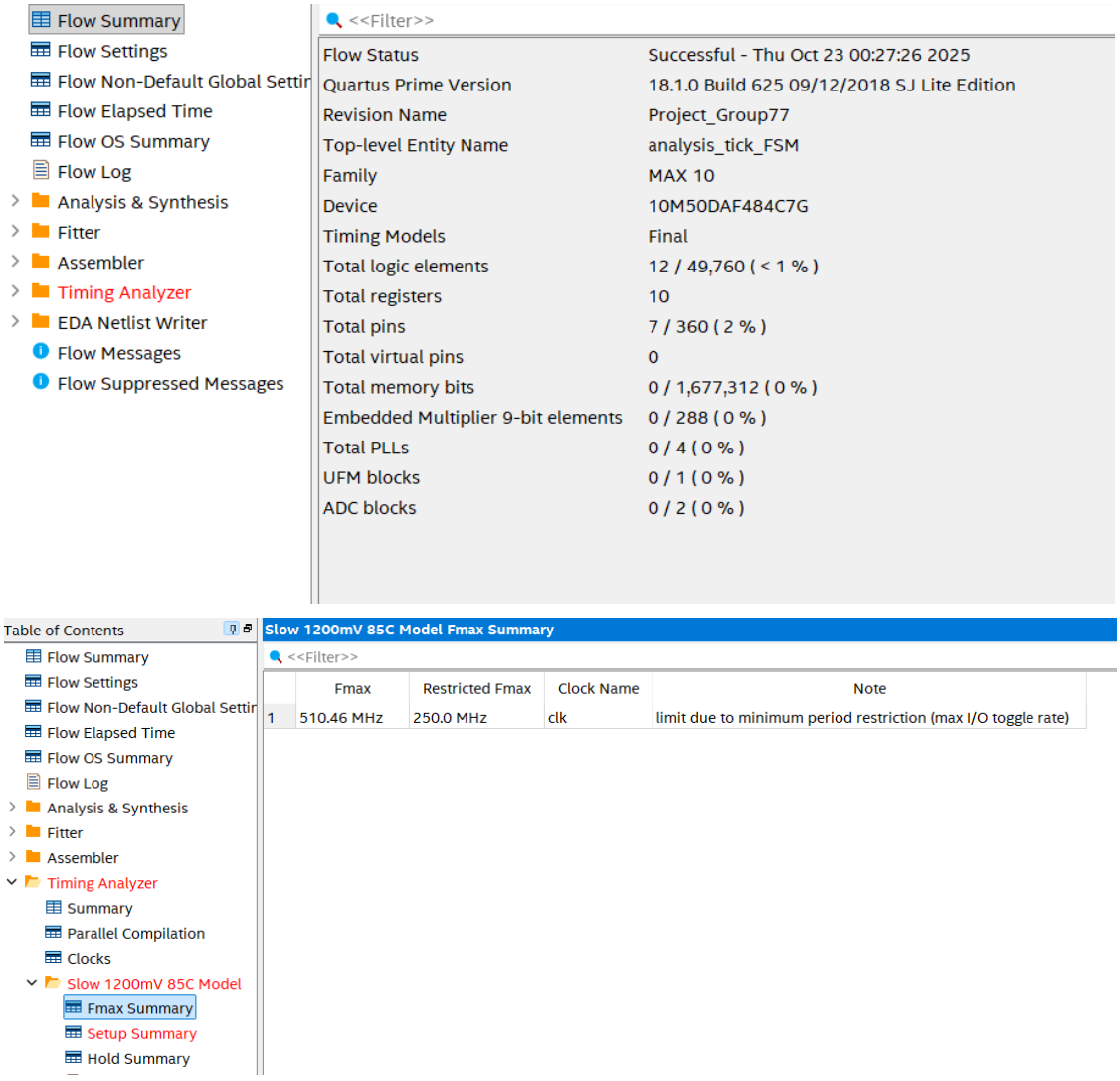
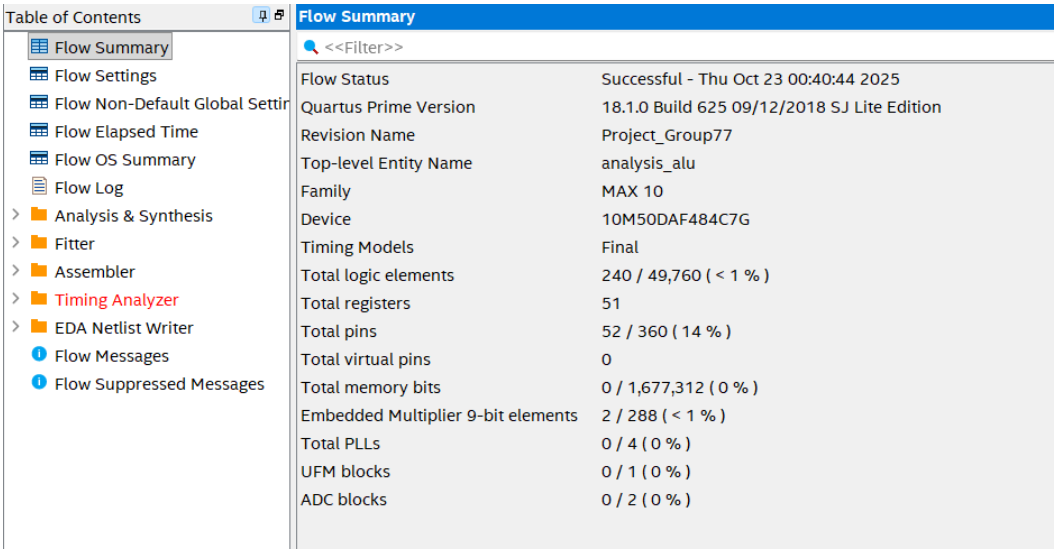


Figure 7.2.2: Logic Usage and Timing Constraints of FSM module

7.2.3 ALU analysis



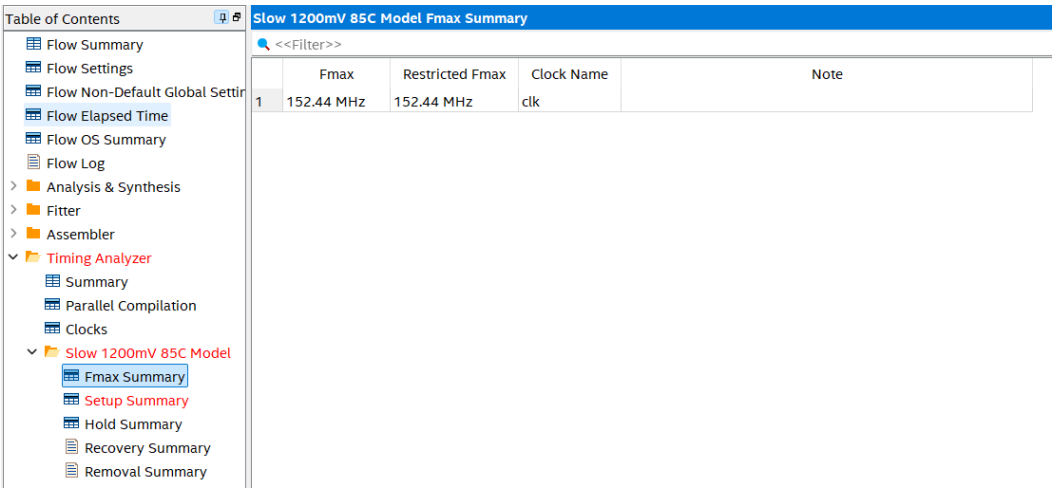
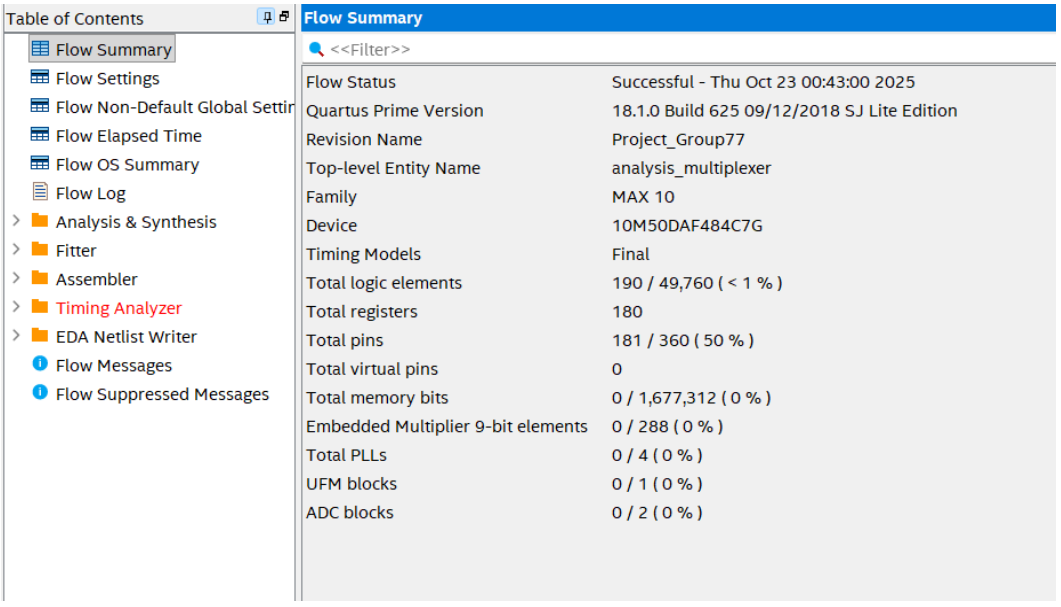


Figure 7.2.3: Logic Usage and Timing Constraints of ALU module

7.2.4 Multiplexer analysis



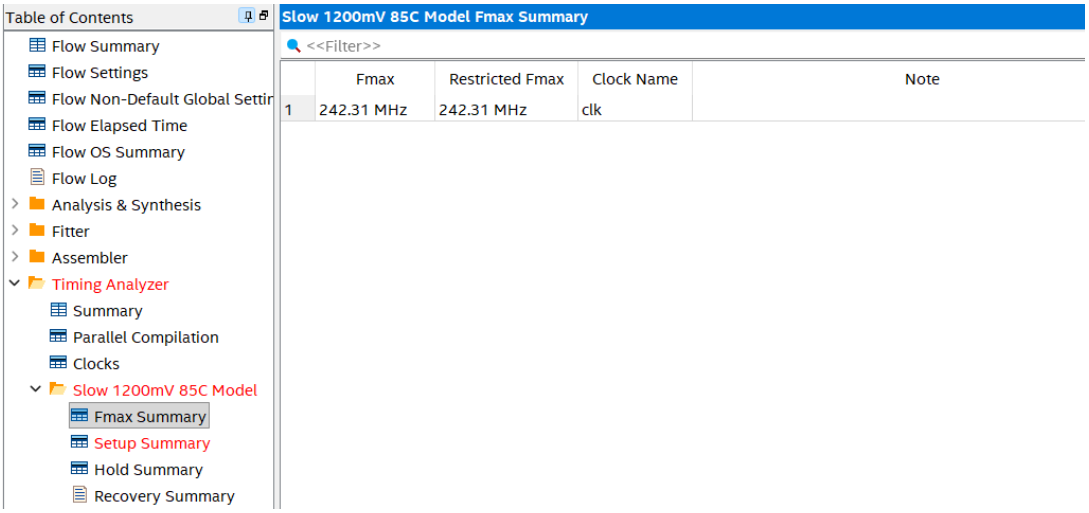


Figure 7.2.4: Logic Usage and Timing Constraints of Multiplexer module

7.2.5 Register Analysis

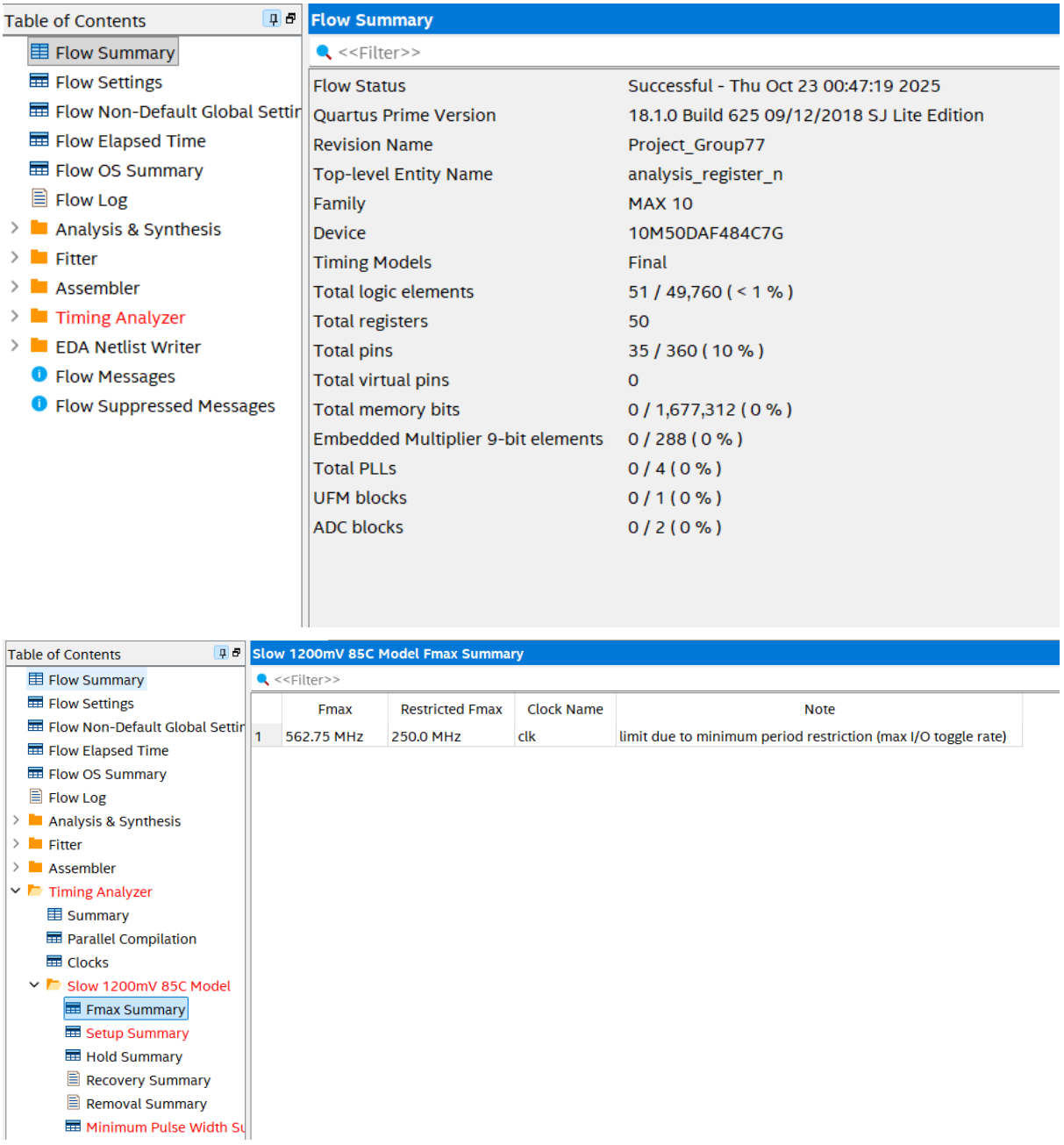


Figure 7.2.5: Logic Usage and Timing Constraints of Register module

7.3 testbench's Transcripts and Waveform

7.3.1 Task 1 testbench

```
VSIM 52> run -all
# Done with test for part 2 - tick FSM.
# 8 tests run.
# There were 0 errors for part 2.
# Done with test for part 3 - multiplexer.
# 16 tests run.
# There were 0 errors for part 3.
# Done with test for part 4 - ALU.
# 128 tests run.
# There were 0 errors for part 4.
# Done with manual computed test for part 4 - ALU.
# 5 manual tests run.
# There were 0 errors for part 4 manual computed test.
# Done with test for part 1 - sign extender.
# 512 tests run.
# There were 0 errors for part 1.
# Done with test for part 5 - register.
# 660 tests run.
# There were 0 errors for part 5.
# SUMMARY:
#  Part 1 - Sign Extender:   0 errors
#  Part 2 - tick_FSM:        0 errors
#  Part 3 - Multiplexer:     0 errors
#  Part 4 - ALU:             0 errors
#  Part 5 - Register:        0 errors
# ** Note: $stop    : C:/Users/jieyi/Downloads/Project_Group77/Project_Group77_restored/components_tb.v(450)
#    Time: 6596 ns  Iteration: 1  Instance: /components_tb
```

Figure 7.3.1.1 Test 1 Testbench Result

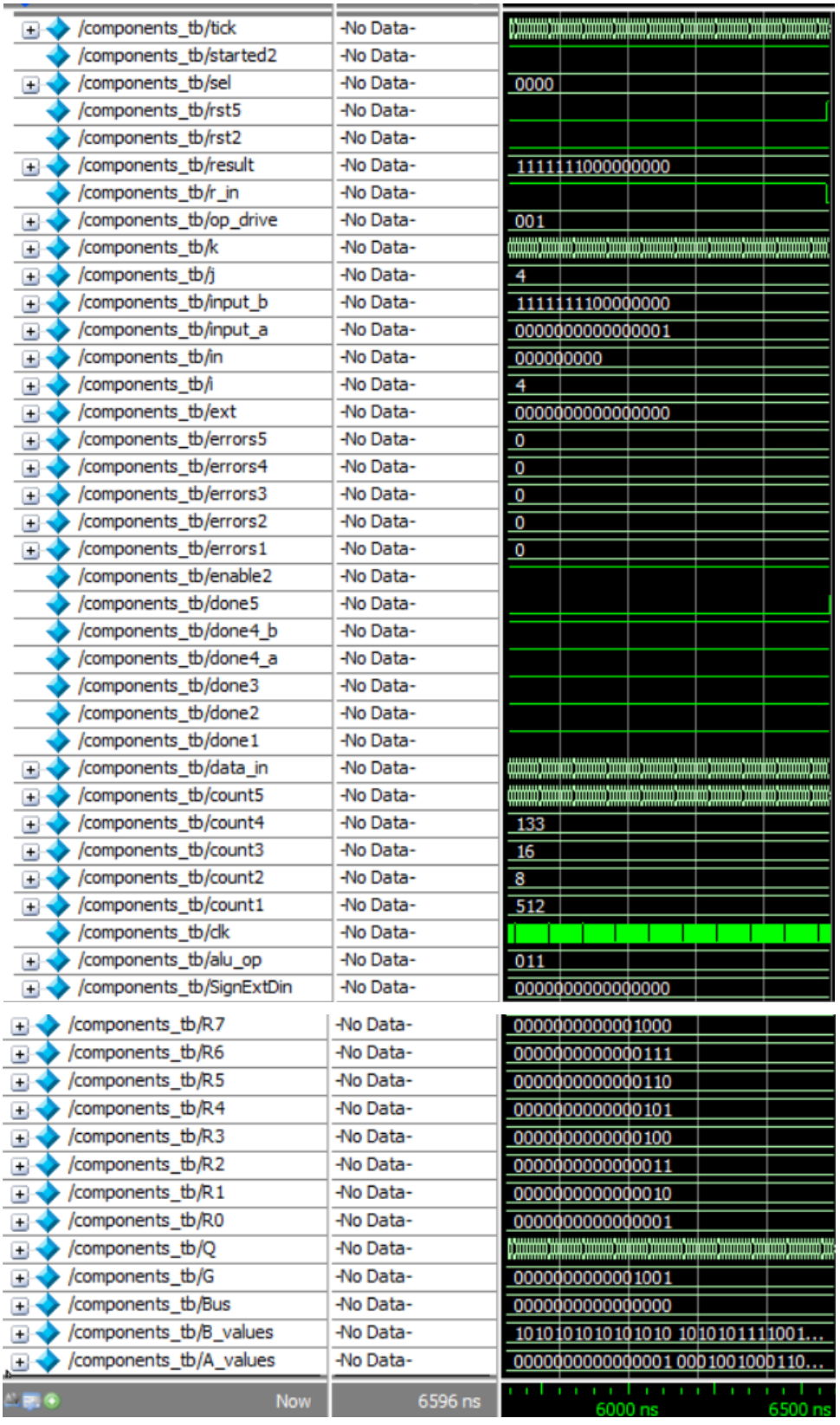


Figure 7.3.1.2: Task 1 Testbench waveform and transcript

7.3.2 Task 2 testbench

```
VSIM 36> run -all
# Done with test for reset. 8 tests run.
# Done with test for movi function loop 0. 8 tests run.
# Done with test for add function loop 0. 64 tests run.
# Done with test for sub function loop 0. 64 test run.
# Done with test for addi function loop 0. 8 tests run.
# Done with test for movi function loop 1. 8 tests run.
# Done with test for add function loop 1. 64 tests run.
# Done with test for sub function loop 1. 64 test run.
# Done with test for addi function loop 1. 8 tests run.
# Done with test for movi function loop 2. 8 tests run.
# Done with test for add function loop 2. 64 tests run.
# Done with test for sub function loop 2. 64 test run.
# Done with test for addi function loop 2. 8 tests run.
# Done with test checking HEX5 display.
# Done with edge case test computed manually. 6 tests run.
# Done with test for task 2.
# 2130 tests run.
# Total errors: 0
# ** Note: $stop    : C:/Users/jieyi/Downloads/Project_Group77/Project_Group77_restored/proc_tb.v(411)
#    Time: 18066 ns  Iteration: 0  Instance: /proc_tb
```

Figure 7.3.2.1 Test 2 Testbench Result

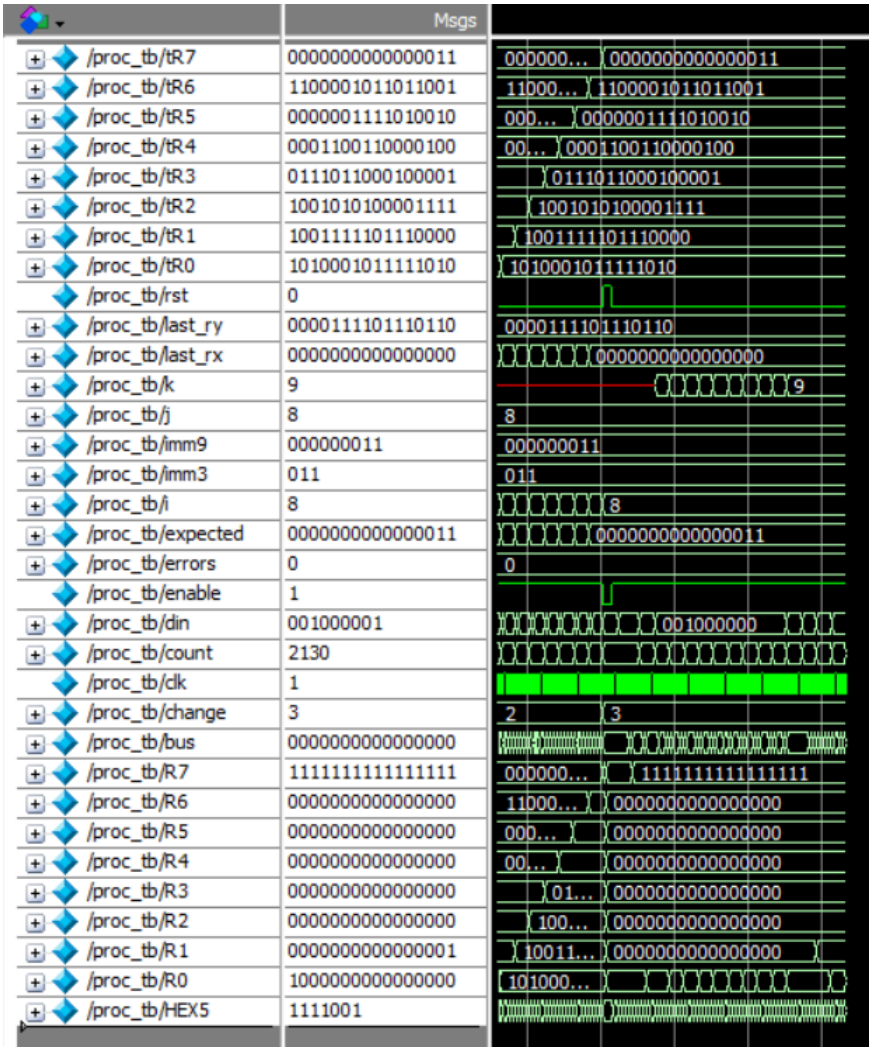


Figure 7.3.2.2: Task 2 Testbench waveform and transcript

7.3.3 Task 3 testbench

```
VSIM 44> run -all
# Done with test for reset. 8 tests run.
# Done with test for movi function loop 0. 8 tests run.
# Done with test for add function loop 0. 64 tests run.
# Done with test for sub function loop 0. 64 tests run.
# Done with test for addi function loop 0. 8 tests run.
# Done with test for mul function loop 0. 64 tests run.
# Done with test for ssi function loop 0. 8 tests run.
# Done with test for disp function loop 0. 8 tests run.
# Done with test for movi function loop 1. 8 tests run.
# Done with test for add function loop 1. 64 tests run.
# Done with test for sub function loop 1. 64 tests run.
# Done with test for addi function loop 1. 8 tests run.
# Done with test for mul function loop 1. 64 tests run.
# Done with test for ssi function loop 1. 8 tests run.
# Done with test for disp function loop 1. 8 tests run.
# Done with test for movi function loop 2. 8 tests run.
# Done with test for add function loop 2. 64 tests run.
# Done with test for sub function loop 2. 64 tests run.
# Done with test for addi function loop 2. 8 tests run.
# Done with test for mul function loop 2. 64 tests run.
# Done with test for ssi function loop 2. 8 tests run.
# Done with test for disp function loop 2. 8 tests run.
# Done with test checking HEX5 display.
# Done with edge case test computed manually. 12 tests run.
# Done with test for task 3.
# 3396 tests run.
# Total errors: 0
# ** Note: $stop    : C:/Users/jieyi/Downloads/Project_Group77/Project_Group77_restored/proc_extension_tb.v(642)
#    Time: 27986 ns  Iteration: 0  Instance: /proc_extension_tb
```

Figure 7.3.3.1 Test 3 Testbench Result

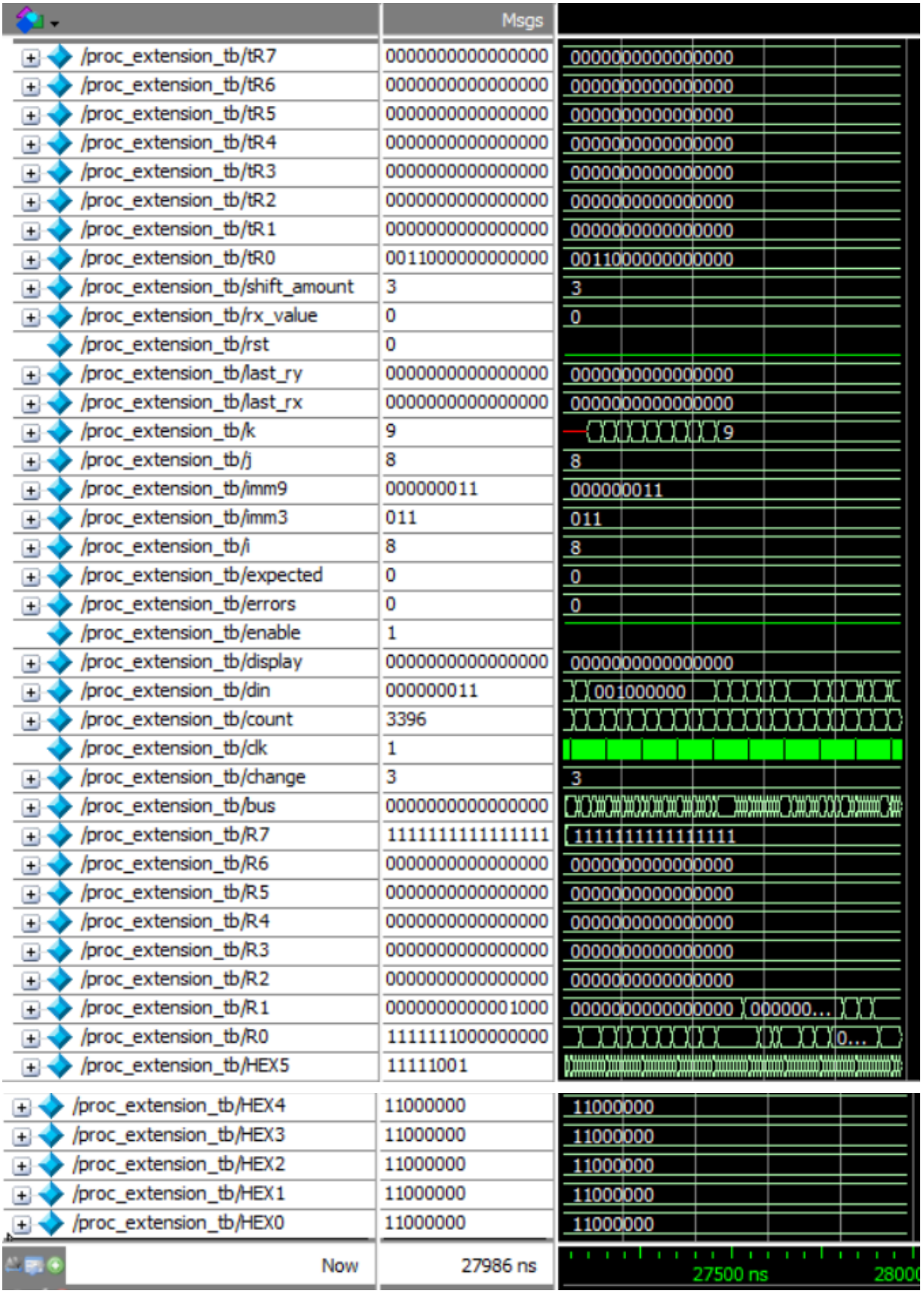


Figure 7.3.3.2: Task 3 Testbench waveform and transcript